

AWS EC2 CD 파이프라인 구축과 배포 자동화



학습 목표

- 지속적 제공과 지속적 배포를 승인 게이트의 유무로 구분하고 채택 기준을 판단합니다
- 단일 EC2 인스턴스에서 Nginx 동적 프록시 전환으로 0초 무중단 배포를 구현합니다
- GitHub Actions SSH 액션으로 CI 성공 직후 자동 실행되는 Push 배포 파이프라인을 구성합니다
- Watchtower 폴링 모드와 웹훅 모드로 서버 내부 Pull 방식 배포 동작을 검증합니다
- Slack Incoming Webhook으로 배포 성공과 실패 결과를 팀 채널로 실시간 전송합니다

CI는 병합 이전에 결함을 차단하고, CD는 검증된 산출물을 사람의 개입 없이 운영 환경까지 전달합니다. 본 차시는 GHCR에 발행된 이미지를 EC2 호스트의 실행 상태로 연결하는 마지막 구간을 자동화합니다.

핵심 개념 및 구조

Continuous Delivery와 Continuous Deployment

비교 항목	Continuous Delivery	Continuous Deployment
최종 릴리스 방식	수동 승인 게이트 통과 후 릴리스	테스트 통과 시 완전 자동 릴리스
자동화 범위	빌드와 테스트, 스테이징 배포까지	프로덕션 릴리스까지 전 구간
선행 요구 조건	릴리스 매니저의 승인 절차	자동 헬스체크와 즉시 롤백 체계
주요 채택 환경	금융, 공공, 의료 등 규제 도메인	SaaS, 스타트업 등 출시 속도 우선

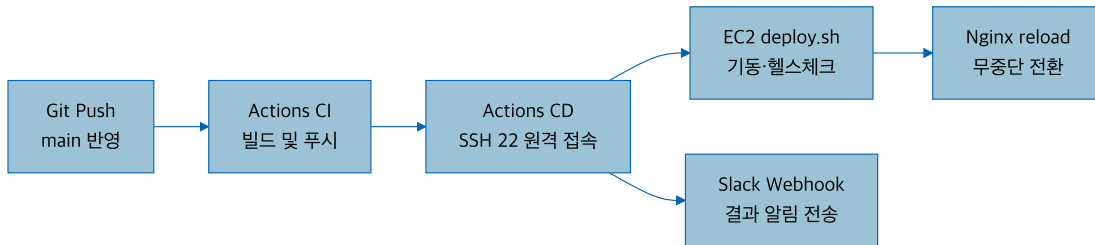
배포(Deployment) vs 릴리스(Release): 배포는 아티팩트를 서버에 설치·기동하는 기술적 작업이며, **릴리스**는 배포된 기능을 사용자에게 공개하는 비즈니스 의사결정입니다. Delivery는 언제든지 릴리스 가능한 준비 상태(Release Ready)를 유지하고 승인 후 릴리스하며, Deployment는 테스트 통과 즉시 릴리스까지 전면 자동화합니다.

배포 아키텍처: Push 방식과 Pull 방식

비교 항목	Push 방식: GitHub Actions SSH	Pull 방식: Watchtower 에이전트
제어 주체	외부 CI 러너가 원격 명령 전달	호스트 내부 상주 데몬 컨테이너
인바운드 포트	SSH 22번 인바운드 허용 필수	불필요, 아웃바운드 통신만 사용
자격 증명 위치	GitHub Secrets에 프라이빗 키 보관	호스트 내부에만 GHCR 자격 증명 보관
배포 로그 가시성	Actions 콘솔에서 실시간 확인	에이전트 로그, 중앙 알림 연동 필요
도커 제어 권한	SSH 계정의 <code>docker</code> 그룹 권한	<code>/var/run/docker.sock</code> 마운트

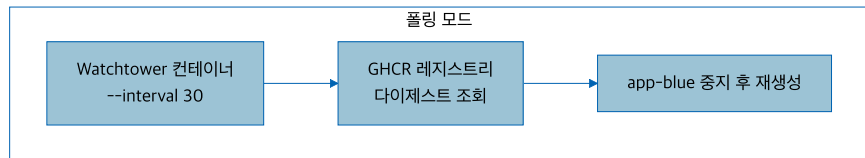
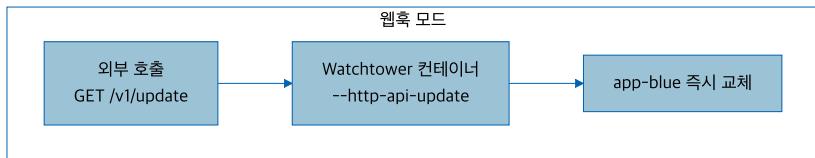
데몬 컨테이너(Daemon Container): 호스트 백그라운드에서 상주하며 `/var/run/docker.sock` 을 통해 도커 엔진을 직접 제어하고 새 이미지를 감지·교체하는 에이전트(Watchtower)입니다. Push 방식은 22번 포트를 여는 대신 가시성이 높고, Pull 방식은 포트를 닫는 대신 데몬 컨테이너에 도커 제어권을 위임합니다.

Push 기반 SSH 무중단 배포 파이프라인



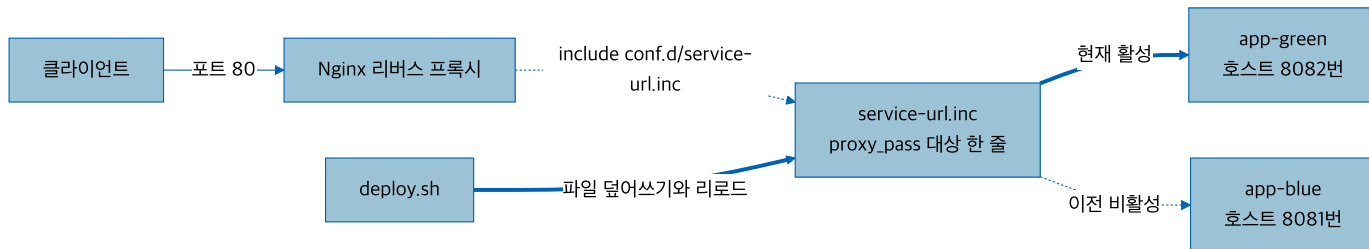
CD 파이프라인은 이미지를 직접 빌드하지 않습니다. CI가 GHCR에 발행한 이미지를 EC2 호스트가 다운로드해 교체하므로, 러너는 배포 명령을 전달하는 역할만 수행합니다.

Watchtower Pull 컨테이너 자동 갱신 구조



구분	트리거 조건	네트워크 비용
폴링 모드	지정 주기마다 레지스트리 다이제스트 질의	변경이 없어도 주기마다 발생
웹훅 모드	Bearer 토큰이 포함된 HTTP 요청 수신 시	유휴 상태에서 발생하지 않음

Nginx 동적 리버스 프록시 트래픽 전환



구성 요소	역할
<code>nginx.conf</code>	<code>include</code> 지시어만 선언하며 배포 중에는 변경되지 않음
<code>service-url.inc</code>	활성 업스트림 한 줄만 담아 배포마다 덮어쓰는 설정 조각

단일 인스턴스 Blue/Green 라이프사이클

단계	핵심 동작	판정 기준
활성 대상 감지	<code>service-url.inc</code> 내용으로 Blue와 Green 판별	비활성 대상이 배포 타겟으로 결정
이미지 다운로드와 기동	GHCR <code>pull</code> 후 비활성 서비스만 <code>up -d</code>	신규 컨테이너가 독립 포트에서 기동
L7 헬스체크	비활성 포트 3초 간격 최대 10회 폴링	<code>HTTP 200</code> 수신 시 통과
프록시 스위칭	설정 조각 갱신 후 <code>nginx -s reload</code>	기존 연결 유지 상태로 대상 전환
구버전 회수	이전 컨테이너 <code>stop</code> 및 미태그 이미지 정리	호스트 메모리와 디스크 회수

헬스체크가 지정 횟수 내에 `200`을 받지 못하면 신규 컨테이너를 즉시 중단하고 프록시 대상을 전환하지 않습니다. 기존 활성 서비스가 그대로 유지되므로 실패한 배포가 사용자에게 노출되지 않습니다.

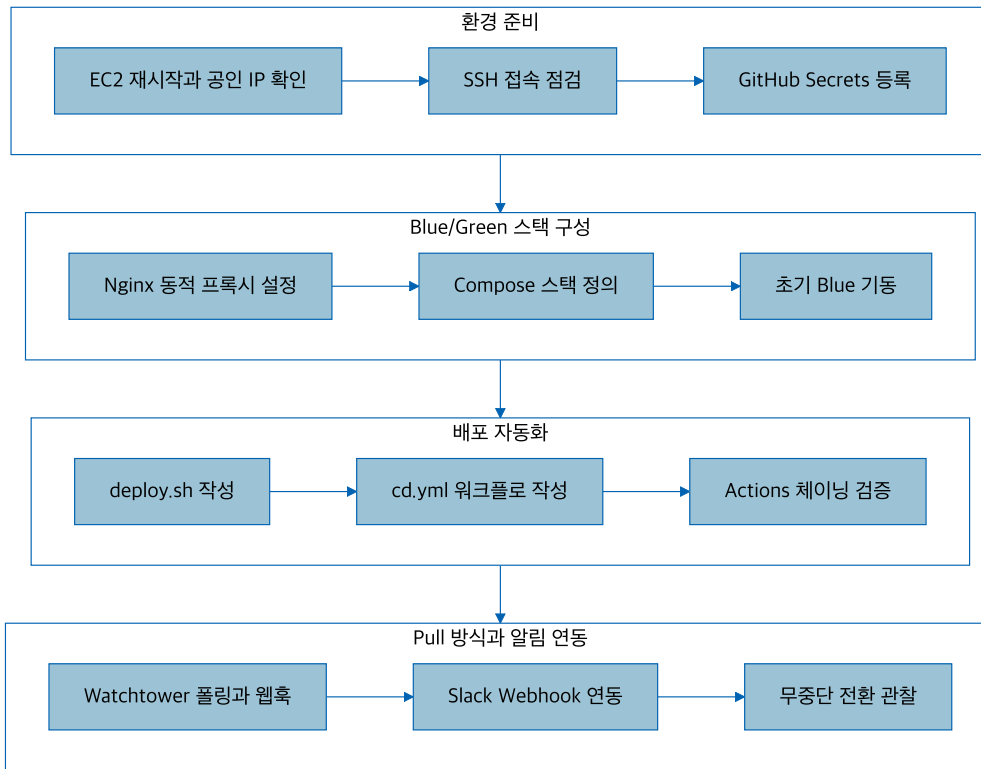
단일 인스턴스와 다중 인스턴스 Blue/Green

비교 항목	단일 인스턴스: Nginx 프록시	다중 인스턴스: ALB Target Group
전환 주체	호스트 내부 Nginx 프로세스	AWS 관리형 L7 로드밸런서
최소 인프라	EC2 인스턴스 1대로 완결	인스턴스 2세트와 ALB 시간당 비용
호스트 부담	전환 구간에 2개 JVM 공존	신규 인스턴스 그룹에 독립 기동
장애 격리	호스트 장애 시 전체 서비스 중단	멀티 AZ 분산으로 고가용성 확보

단일 Target Group에 인스턴스들을 등록한 기본 라운드로빈 구조와 달리, Target Group을 2개로 나누어 가중치를 전환하는 다중 인스턴스 Blue/Green은 심화 아키텍처 모델입니다. 본 실습에서는 단일 인스턴스 내 Nginx 리버스 프록시 동적 스위칭을 직접 구현합니다.

실습 단계

실습 전체 흐름: EC2 재기동부터 무중단 전환 검증까지



EC2 인스턴스 재시작과 공인 IP 확인

```
export STUDENT_ID="student01" AWS_PAGER=""
export AWS_PROFILE="$STUDENT_ID" AWS_REGION="ap-northeast-2"
export MY_KEY_NAME="${STUDENT_ID}-key" MY_SG_NAME="${STUDENT_ID}-web-sg"
export MY_INSTANCE_NAME="${STUDENT_ID}-managed-ec2"
cd ~/workspace/aws-lab          # .pem 보관 디렉터리
export INSTANCE_ID=$(aws ec2 describe-instances \
  --filters "Name=tag:Name,Values=$MY_INSTANCE_NAME" \
    "Name=instance-state-name,Values=stopped" \
  --query "Reservations[0].Instances[0].InstanceId" \
  --output text)

aws ec2 start-instances --instance-ids "$INSTANCE_ID"
aws ec2 wait instance-status-ok --instance-ids "$INSTANCE_ID"
export PUBLIC_IP=$(aws ec2 describe-instances \
  --instance-ids "$INSTANCE_ID" \
  --query "Reservations[0].Instances[0].PublicIpAddress" \
  --output text); echo "$PUBLIC_IP"
```

명령어	기능
<code>STUDENT_ID</code> 파생	사용자명으로 자원 이름 생성
이름·상태 필터	타 수강생 인스턴스 오조회 방지
<code>wait instance-status-ok</code>	2/2 상태 검사 통과 대기
<code>describe-instances --query</code>	신규 공인 IP만 추출



공인 IP 재할당

탄력적 IP가 없는 인스턴스는 재시작할 때마다 공인 IP가 바뀌므로 접속 정보를 매번 갱신합니다

SSH 접속 사전 점검과 프라이빗 키 복사

```
ssh -i ./"$MY_KEY_NAME".pem \  
-o StrictHostKeyChecking=accept-new \  
ubuntu@"$PUBLIC_IP" "sudo docker compose version"  
  
pbcopy < ./"$MY_KEY_NAME".pem
```

Windows Git Bash에서는 `clip`, 리눅스에서는 `xclip -selection clipboard`로 대체합니다.

옵션	기능
<code>-i ./*.pem</code>	키 페어 인증용 프라이빗 키 지정
<code>StrictHostKeyChecking=accept-new</code>	신규 호스트 키를 프롬프트 없이 수락
<code>"sudo docker compose version"</code>	접속과 도커 가용성을 한 번에 점검

🔑 키 전문 복사

`-----BEGIN` 부터 `-----END` 까지 개행을 포함한 전체
내용을 그대로 옮겨야 러너가 인증에 성공합니다

GitHub Repository Secrets 등록

1. `simple-back-ghcr` 저장소의 `Settings` 메뉴로 이동
2. `Secrets and variables` → `Actions` 선택
3. `New repository secret` 클릭 후 3건을 각각 등록
4. 등록 후에는 값을 다시 조회할 수 없으므로 오타를 즉시 확인

암호화 저장

등록된 값은 저장소 설정에 암호화되어 보관되며 실행 로그에 노출될 경우 자동으로 마스킹 처리됩니다

시크릿 이름	값	용도
<code>EC2_HOST</code>	재시작 후 공인 IP	접속 대상 호스트
<code>EC2_USER</code>	<code>ubuntu</code>	Ubuntu AMI 기본 계정
<code>EC2_SSH_KEY</code>	pem 키 전문	공개키 인증 자격 증명

프라이빗 키를 저장소 파일이나 워크플로 파일에 직접 기록하면 이력에 영구히 남습니다. 자격 증명은 반드시 Secrets를 통해서만 주입합니다.

GitHub Actions 러너의 SSH 인바운드 허용

```
export MY_SG_ID=$(aws ec2 describe-security-groups \
  --filters "Name=group-name,Values=$MY_SG_NAME" \
  --query "SecurityGroups[0].GroupId" --output text)

export RUNNER_SSH_RULE_ID=$(aws ec2 \
  authorize-security-group-ingress \
  --group-id "$MY_SG_ID" --protocol tcp \
  --port 22 --cidr 0.0.0.0/0 \
  --query "SecurityGroupRules[0].SecurityGroupRuleId" \
  --output text)
```

항목	내용
러너 공인 IP	실행마다 바뀌는 동적 주소
기존 22번 규칙	학습자 IP <code>/32</code> 만 허용
규칙 미추가 시	<code>dial tcp *:22: i/o timeout</code>



실습용 임시 개방

키 인증만 허용되는 전제의 임시 조치이며 정리 단계에서 `RUNNER_SSH_RULE_ID` 규칙만 회수합니다. 실무에서는 22번 포트를 열지 않는 SSM Session Manager나 Self-hosted Runner를 씁니다.

Nginx 동적 프록시 설정 파일 작성

```
events { worker_connections 1024; }

http {
    server {
        listen 80;
        location / {
            include /etc/nginx/conf.d/service-url.inc;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
        }
    }
}
```

```
proxy_pass http://app-blue:8080;
```

N 설정 분리의 목적

배포 스크립트는 위 한 줄짜리 `service-url.inc` 만
덮어쓰고, 메인 설정 파일 `nginx.conf` 는 전혀 건드리지
않습니다

두 파일 모두 컨테이너에 읽기 전용(`ro`)으로 마운트하며,
호스트 파일을 수정한 뒤 리로드 신호를 보내면 즉시
반영됩니다.

Blue/Green Compose 스택 정의

```
services:
  nginx:
    image: nginx:alpine
    container_name: nginx-proxy
    ports: ["80:80"]
    volumes:
      - ./nginx.conf:/etc/nginx/nginx.conf:ro
      - ./service-url.inc:/etc/nginx/conf.d/service-url.inc:ro
    networks: [frontend-net]
  app-blue:
    image: ghcr.io/${GITHUB_USERNAME}/simple-back-ghcr:latest
    container_name: app-blue
    ports: ["8081:8080"]
    env_file: [.env]
    environment: { JAVA_TOOL_OPTIONS: "-XX:MaxRAMPercentage=75.0" }
    deploy: { resources: { limits: { memory: 896M } } }
    networks: [frontend-net, backend-net]
# app-green: app-blue와 동일하고 이름과 포트만 다름
app-green: { container_name: app-green, ports: ["8082:8080"] }
```

항목	값
Blue 포트	8081
Green 포트	8082
앱 메모리 상한	896M
~/.env 키	DB_*, MYSQL_USER · MYSQL_PASSWORD



메모리 상한과 Green 정의

t4g.small (2GB)에서 두 JVM이 공존해도 OOM이 없도록 상한을 둡니다.
app-green은 이름과 호스트 포트만 다릅니다.

초기 Blue 스택 기동과 응답 점검

```
sudo docker compose down 2>/dev/null || true

sudo docker compose -f ~/compose.blue-green.yaml \
  up -d db nginx app-blue
sleep 10

curl -s http://localhost/
curl -s http://localhost:8081/

sudo docker compose -f ~/compose.blue-green.yaml ps
```

점검 경로

검증 대상

`http://localhost/`

Nginx 80번 진입점 라우팅

`http://localhost:8081/`

Blue 컨테이너 직접 응답

`compose ps`

3개 서비스 기동 상태

☑ 502 응답 시 확인

스프링부트가 DB 커넥션을 맺는 중이면 일시적으로 502가 반환됩니다. `sudo docker logs -f --tail 30 app-blue` 로 기동 로그를 확인합니다

배포 스크립트의 활성화 대상 감지와 신규 컨테이너 기동

```
#!/usr/bin/env bash
set -euo pipefail
cd ~/

CURRENT_URL=$(cat ~/service-url.inc)

if echo "$CURRENT_URL" | grep -q "app-blue"; then
    CURRENT_SERVICE="app-blue"; CURRENT_PORT=8081
    TARGET_SERVICE="app-green"; TARGET_PORT=8082
else
    CURRENT_SERVICE="app-green"; CURRENT_PORT=8082
    TARGET_SERVICE="app-blue"; TARGET_PORT=8081
fi

sudo docker compose -f compose.blue-green.yaml pull "$TARGET_SERVICE"
sudo docker compose -f compose.blue-green.yaml up -d "$TARGET_SERVICE"
```

구문	기능
<code>set -euo pipefail</code>	실패 시 즉시 중단
<code>grep -q "app-blue"</code>	현재 활성화 대상 판별
<code>pull</code> <code>"\$TARGET_SERVICE"</code>	비활성 대상만 이미지 다운로드
<code>up -d</code> <code>"\$TARGET_SERVICE"</code>	활성 서비스 무중단 유지

🖼️ 상태의 단일 출처

배포 대상 판별의 기준은 오직 `service-url.inc` 파일 한 줄이므로 스크립트가 별도의 상태 저장소를 두지 않습니다

헬스체크 폴링과 Nginx 무중단 스위칭

```
HEALTH_OK=false
for i in $(seq 1 10); do
    STATUS=$(curl -s -o /dev/null -w "%{http_code}" \
        "http://localhost:$TARGET_PORT/" || true)
    if [ "$STATUS" -eq 200 ]; then HEALTH_OK=true; break; fi
    sleep 3
done

if [ "$HEALTH_OK" = false ]; then
    sudo docker compose -f compose.blue-green.yaml stop "$TARGET_SERVICE"
    exit 1
fi

echo "proxy_pass http://$TARGET_SERVICE:8080;" > ~/service-url.inc
sudo docker exec nginx-proxy nginx -s reload

sudo docker compose -f compose.blue-green.yaml stop "$CURRENT_SERVICE"
sudo docker image prune -f
```

단계	동작
헬스체크 폴링	3초 간격 최대 10회 상태 코드 확인
실패 처리	신규 컨테이너 중단 후 종료 코드 1
프록시 전환	설정 조각 갱신 후 리로드 신호 전송
자원 회수	구버전 중지 및 미태그 이미지 정리

🔄 리로드와 재시작의 차이

`nginx -s reload`는 마스터 프로세스를 유지한 채 새 워커를 띄우므로 처리 중인 연결이 끊기지 않습니다

배포 스크립트 수동 전환 검증

```
chmod +x ~/deploy.sh
~/deploy.sh

cat ~/service-url.inc
sudo docker compose -f ~/compose.blue-green.yaml ps -a

exit
```

확인 대상	기대 결과
스크립트 출력	헬스체크 통과 후 전환 완료 로그
<code>service-url.inc</code>	<code>app-green</code> 으로 대상 갱신
<code>compose ps -a</code>	Green 실행, Blue 중지(<code>Exited</code>) 상태

☐ 자동화 이전의 수동 검증

워크플로에 연결하기 전에 호스트에서 직접 실행해 두면, 이후 배포 실패의 원인을 스크립트와 파이프라인 중 하나로 좁힐 수 있습니다

simple-back-ghcr 에 테스트 게이트 CI 도입

```
# 테스트 없이 곧바로 발행하던 워크플로 제거
git rm .github/workflows/docker-publish.yml
# 빈 디렉터리도 함께 사라지므로 재생성
mkdir -p .github/workflows

# 08에서 익힌 test → docker-build-push
# needs 의존 패턴을 이 저장소에 적용
cat <<'EOF' > .github/workflows/ci.yml
name: Continuous Integration & Package
# test Job → docker-build-push Job
# (needs: test, platforms: linux/arm64,
# runs-on: ubuntu-24.04-arm, cache: gha)
EOF
```

대상	이전	이후
워크플로 파일	<code>docker- publish.yml</code>	<code>ci.yml</code>
발행 조건	테스트 없이 즉시 발행	<code>test</code> 성공 시에만 발행

💡 08 패턴의 재적용

`08`은 별도 학습용 프로젝트(`spring-test-ci`)에서 이 패턴을 개념적으로 익혔을 뿐, 실제 배포 대상(`simple-back-ghcr`)에는 이 Step에서 처음 적용합니다.

CD 워크플로 트리거와 실행 조건

```
name: CD Pipeline (Push via SSH)

on:
  workflow_run:
    workflows: ["Continuous Integration & Package"]
    types: [completed]
    branches: [main]
  workflow_dispatch:

jobs:
  deploy:
    name: Blue-Green Deploy to AWS EC2
    runs-on: ubuntu-latest
    if: ${{ github.event_name == 'workflow_dispatch' || github.event.workflow_run.conclusion == 'success' }}
```

`workflow_run`은 선행 워크플로의 완료 이벤트를 받아 실행되며, 완료에는 실패도 포함됩니다. 따라서 `conclusion`이 `success`인 경우로 조건을 좁혀야 실패한 빌드의 배포를 차단할 수 있습니다.

SSH 원격 배포 스텝 정의

```
steps:
  - name: Blue/Green 무중단 배포 실행
    id: ssh_deploy
    uses: appleboy/ssh-action@v1.2.0
    with:
      host: ${ secrets.EC2_HOST }
      username: ${ secrets.EC2_USER }
      key: ${ secrets.EC2_SSH_KEY }
      port: 22
      script: |
        ~/deploy.sh
```

설정 키	기능
<code>uses</code>	검증된 원격 SSH 실행 액션 지정
<code>id</code>	후속 스텝이 결과를 참조할 식별자
<code>key</code>	Secrets에서 주입되는 프라이빗 키
<code>script</code>	호스트에서 비대화형으로 실행할 명령

✓ 러너의 유동 IP

관리형 러너는 접속 IP가 매번 바뀌므로 보안 그룹의 SSH 인바운드 범위를 어떻게 통제할지가 Push 방식의 핵심 과제입니다

CD 워크플로 푸시와 Actions 체이닝 확인

```
git add .github/workflows/cd.yml
git commit -m "ci: add SSH Blue/Green CD pipeline"
git push origin main
```

🔗 워크플로 체이닝

CD 워크플로는 푸시 자체가 아니라 선행 CI 워크플로의 완료 이벤트에 반응하므로 CI 종료 후에 실행이 시작됩니다

확인 단계	위치	기대 결과
원격 반영	터미널 출력	<code>origin main</code> 정상 푸시
CI 실행	Actions 콘솔	테스트 통과와 GHCR 발행
CD 실행	Actions 콘솔	체이닝 감지 후 러너 프로비저닝

CD 워크플로 파일은 기본 브랜치에 존재해야 `workflow_run` 트리거가 등록됩니다. 첫 실행은 병합 이후 발생한 CI 완료 이벤트부터 적용됩니다.

원격 호스트 전환 상태와 서비스 응답 검증

```
ssh -i ~/.ssh/$MY_KEY_NAME.pem ubuntu@$PUBLIC_IP" \  
"cat ~/service-url.inc && \  
sudo docker compose -f ~/compose.blue-green.yaml ps"  
  
curl -s http://"$PUBLIC_IP"/
```

검증 항목	확인 내용
라우팅 대상	설정 조각이 신규 서비스로 갱신
컨테이너 상태	신규 실행, 구버전 중지
외부 응답	공인 IP 80번 포트 정상 응답

HTTP 배포 로그 추적

Actions 실행 화면에서 배포 스텝을 펼치면 헬스체크 시도 횟수와 프록시 전환 시점이 그대로 출력됩니다

Watchtower 폴링 모드 구동과 다이제스트 감시

```
ssh -i ./"$MY_KEY_NAME".pem ubuntu@"$PUBLIC_IP"
export GITHUB_USERNAME=본인_깃허브_아이디 CR_PAT=ghp_발급토큰

sudo docker run -d --name watchtower-poll \
  --restart unless-stopped \
  -v /var/run/docker.sock:/var/run/docker.sock \
  -e DOCKER_API_VERSION=1.44 \
  -e REPO_USER="$GITHUB_USERNAME" -e REPO_PASS="$CR_PAT" \
  containrrr/watchtower --interval 30 --cleanup app-blue
sudo docker logs -f --tail 20 watchtower-poll
```

옵션	기능
<code>docker.sock</code> 마운트	호스트 도커 제어권 위임
<code>DOCKER_API_VERSION</code>	Docker 29 호환 API 지정
<code>REPO_USER</code> · <code>REPO_PASS</code>	비공개 GHCR 인증
<code>--interval 30</code>	30초 주기 다이제스트 질의
<code>app-blue</code>	감시 대상 컨테이너 한정



다이제스트 비교

태그가 같아도 다이제스트가 바뀌면 새 빌드로 판정하고, `--cleanup`이
구버전을 정리합니다. API 버전을 빼면 `client version 1.25 is too old`
로 재시작을 반복합니다.

Watchtower HTTP Webhook 모드와 즉시 갱신

```
sudo docker rm -f watchtower-poll
sudo docker run -d --name watchtower-webhook \
  --restart unless-stopped -p 8000:8080 \
  -v /var/run/docker.sock:/var/run/docker.sock \
  -e DOCKER_API_VERSION=1.44 \
  -e REPO_USER="$GITHUB_USERNAME" -e REPO_PASS="$CR_PAT" \
  containrrr/watchtower --http-api-update \
  --http-api-token watchtower-secret-token --cleanup app-blue
curl -s -X GET -H "Authorization: Bearer watchtower-secret-token" \
  http://localhost:8000/v1/update
sudo docker logs --tail 30 watchtower-webhook
```

옵션	기능
<code>-p 8000:8080</code>	웹훅 리스너를 호스트 8000번에 매핑
<code>--http-api-update</code>	주기 폴링 중단, 요청 수신 시에만 동작
<code>--http-api-token</code>	비인가 호출 차단용 Bearer 토큰

`/v1/update` 호출 즉시 GHCR에서 새 이미지를 다운로드해 교체합니다. 토큰 없이 구동하면 8000번 포트로 누구나 배포를 트리거할 수 있으므로 `--http-api-token` 인증이 필수입니다.

폴링 모드와 웹훅 모드의 동작 차이

비교 항목	폴링 모드	웹훅 모드
트리거 주체	Watchtower 자체 타이머	외부 HTTP 요청
평시 네트워크 사용	주기마다 레지스트리 질의 발생	유휴 시 트래픽 없음
반영 지연	최대 폴링 주기만큼 지연	호출 즉시 착수
인바운드 포트	불필요	수신 포트 개방 필요

⇔ 폴링은 인바운드 개방 없이 동작하는 대신 반영이 지연되고, 웹훅은 즉시 반영되는 대신 수신 포트와 토큰 관리가 필요합니다. 실습에서는 두 모드를 모두 구동해 트리거 경로의 차이를 확인합니다.

Watchtower 정리와 Push 파이프라인 복귀

```
sudo docker rm -f watchtower-webhook  
  
sudo docker compose -f ~/compose.blue-green.yaml ps  
  
exit
```

☞ 제거 이유

Watchtower가 남아 있으면 Blue/Green 전환 도중 컨테이너를 임의로 재생성해 배포 결과가 어긋납니다.

명령어

기능

```
sudo docker rm -f
```

```
watchtower-webhook
```

구동 중인 에이전트 강제 중지 및 삭제

```
sudo docker compose ps
```

`app`, `db`, `nginx` 만
남았는지 확인

```
exit
```

원격 SSH 세션 종료 후
로컬 복귀

- Pull 방식 검증을 마쳤으므로 이후 실습은 다시 Push 방식 파이프라인으로 진행합니다.

Slack Incoming Webhook 앱 생성과 URL 발급

1. Slack API 웹사이트에서 **Create New App** → **From scratch**를 선택합니다
2. App Name에 **Deploy-Bot**을 입력하고 실습 워크스페이스를 지정합니다
3. **Features** → **Incoming Webhooks**에서 토글을 **On**으로 활성화합니다
4. **Add New Webhook to Workspace**를 클릭해 수신 채널을 지정합니다
5. 발급된 **Webhook URL**을 클립보드에 복사합니다

`https://api.slack.com/apps`

설정 항목	권장값
App Name	Deploy-Bot
Incoming Webhooks	On
Target Channel	#general
Webhook URL	https://hooks.slack.com/services/ ...

URL 자체가 자격 증명

Webhook URL에는 채널 식별 토큰이 포함되어 있어 유출되면 누구나 채널에 메시지를 보낼 수 있습니다. 저장소나 코드에 남기지 않고 시크릿으로만 관리합니다.

Slack Webhook 시크릿 등록과 SSH 배포 Job

```
name: CD Pipeline (Push via SSH)
on:
  workflow_run: { workflows: ["Continuous Integration & Package"], types: [completed], branches: [main] }
  workflow_dispatch:
jobs:
  deploy:
    runs-on: ubuntu-latest
    if: ${ github.event_name == 'workflow_dispatch' || github.event.workflow_run.conclusion == 'success' }
    steps:
      - name: SSH 원격 무중단 배포 실행
        id: ssh_deploy
        uses: appleboy/ssh-action@v1.2.0
        with:
          host: ${ secrets.EC2_HOST }
          username: ${ secrets.EC2_USER }
          key: ${ secrets.EC2_SSH_KEY }
          port: 22
          script: ~/deploy.sh
```

설정 키	기능
<code>SLACK_WEBHOOK_URL</code>	Secrets에 등록하는 웹훅 수신 URL
<code>id: ssh_deploy</code>	후속 알림 스텝이 배포 결과를 참조할 식별자

자격 증명과 알림의 분리

EC2 접속용 SSH 키와 Slack 웹훅 URL은 모두 GitHub Secrets로 안전하게 분리 주입합니다.

Slack 알림 스텝과 배포 결과 페이로드

```
- name: Slack 배포 결과 알림 전송
  if: always()
  run: |
    STATUS="${{ steps.ssh_deploy.outcome }}"
    [ "$STATUS" = "success" ] && COLOR="#36a64f" || COLOR="#dc3545"
    PAYLOAD=$(cat <<JSON
    {"attachments":[{"color":"$COLOR","title":"EC2 배포 $STATUS",
      "fields":[{"title":"Repo","value":"${{ github.repository }}",
        {"title":"SHA","value":"${{ github.sha }}}"}]}]}
    JSON
    )
    curl -H "Content-Type: application/json" -X POST \
      -d "$PAYLOAD" "${{ secrets.SLACK_WEBHOOK_URL }}"
```

워크플로 키

기능

if:

always()

배포 스텝 실패 시에도 알림 발송 보장

outcome

배포 결과에 따라 **success** / **failure**
반환

📧 조건부 상태 전송

`steps.ssh_deploy.outcome`을 판별하여 성공 시 초록색(**#36a64f**), 실패 시 빨간색(**#dc3545**) 카드로 알림을 전송합니다.

슬랙 연동 워크플로 푸시와 알림 수신 검증

```
git add .github/workflows/cd.yml
git commit -m "ci: add Slack deployment notification"
git push origin main
```

🔔 실행 순서

`Continuous Integration & Package`가 성공으로 종료되면 `workflow_run` 이벤트가 발생해 CD 파이프라인이 이어서 실행됩니다.

점검 항목	확인 채널	기대 결과
워크플로 실행	GitHub Actions 콘솔	전 스텝 정상 통과
알림 전송 응답	알림 스텝 로그	<code>ok</code> 응답 반환
메시지 수신	Slack 지정 채널	성공 카드 렌더링

수신된 카드에서 Repository, Branch, Triggered By, Commit SHA 메타데이터를 확인합니다.

애플리케이션 v2.0 수정과 트래픽 모니터링 루프

```
cd ~/workspace/simple-back

NEW_MSG='Hello, Blue-Green CD v2.0!'
sed -i.bak "s|message: .*|message: \"\${APP_MESSAGE:$NEW_MSG}\"|" \
  src/main/resources/application.yml

./gradlew test
```

✓ 로컬 사전 검증

푸시 전 로컬 테스트를 통과시켜 CI 단계의 실패를 미리 차단합니다.

```
export PUBLIC_IP=확인한_EC2_공인_IP

while true; do
  curl -s -w "%{http_code}\n" \
    http://"$PUBLIC_IP"/
  sleep 0.5
done
```

별도 터미널 창

배포 작업 터미널과 분리된 창에서 0.5초 주기로 요청을 보내 끊김과 502 응답 발생 여부를 감시합니다.

전체 파이프라인 트리거와 무중단 전환 관찰

```
git add src/main/resources/application.yml
git commit -m "feat: upgrade message to v2.0"
git push origin main
```

```
{"message":"Hello Spring Boot 4 Infra ...",...} | HTTP 200
{"message":"Hello Spring Boot 4 Infra ...",...} | HTTP 200
{"message":"Hello, Zero-Downtime ... v2.0!",...} | HTTP 200
{"message":"Hello, Zero-Downtime ... v2.0!",...} | HTTP 200
```

시점	무중단 판정 기준
배포 전	구버전 메시지와 HTTP 200 유지
신규 컨테이너 기동 중	구버전 응답이 끊김 없이 지속 서빙
Nginx 리로드 순간	연결 유실 0건, 다음 요청부터 신버전 응답

 Git Push부터 Slack 알림까지 사람의 개입 없이 이어지는 전 구간을 한 번의 커밋으로 관찰합니다.

E2E 파이프라인 최종 점검

파이프라인 단계	정상 동작 기준
CI 테스트	단위·슬라이스 테스트 통과
이미지 레지스트리	<code>latest</code> 다이제스트 갱신 완료
CD 원격 배포	SSH 접속과 헬스체크 전량 통과

🔍 원격 상태 조회

모니터링 루프를 `Ctrl+C`로 종료한 뒤 `cat ~/service-url.inc`와 `sudo docker compose ps`를 한 번의 SSH 명령으로 확인합니다.

파이프라인 단계	정상 동작 기준
무중단 전환	오류 응답 0건으로 신버전 전환
리버스 프록시	<code>service-url.inc</code> 대상 갱신 확인
팀 메신저 알림	성공 카드와 Commit SHA 수신

✚ 판정의 기준점

배포 성공 여부는 컨테이너 기동 여부가 아니라 L7 응답 코드와 프록시 라우팅 대상의 일치로 판정합니다.

실습 리소스 정리와 비용 통제

```
aws ec2 revoke-security-group-ingress \
  --group-id "$MY_SG_ID" \
  --security-group-rule-ids "$RUNNER_SSH_RULE_ID"

aws ec2 stop-instances --instance-ids "$INSTANCE_ID"
aws ec2 wait instance-stopped --instance-ids "$INSTANCE_ID"

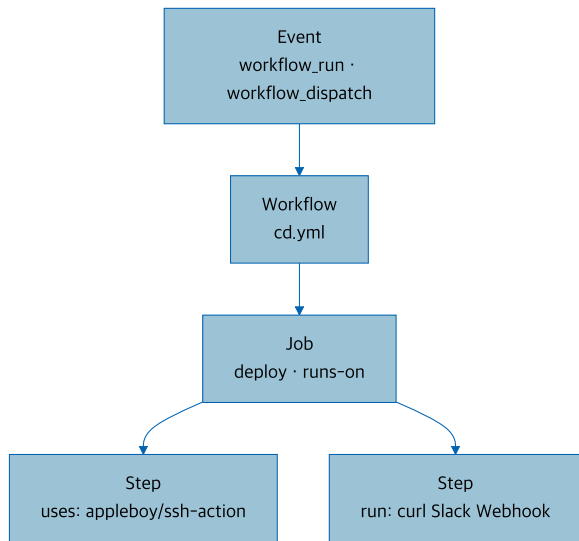
aws ec2 describe-instances \
  --instance-ids "$INSTANCE_ID" \
  --query "Reservations[0].Instances[0].State.Name" \
  --output text
```

명령어	기능
<code>revoke-security-group-ingress</code>	러너용 22번 임시 규칙 회수
<code>stop-instances</code>	vCPU·메모리 과금 즉시 차단
<code>wait instance-stopped</code>	완전 정지까지 대기
<code>describe-instances</code>	상태가 <code>stopped</code> 인지 최종 검증

Compose 볼륨과 설정 파일, GitHub Secrets는 EBS 스토리지와 저장소에 보존되므로 재기동 후 같은 환경으로 이어서 실습할 수 있습니다.

Appendix

GitHub Actions 워크플로 계층과 핵심 키워드



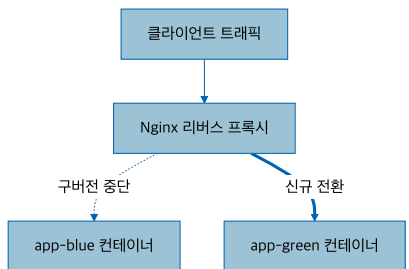
키워드	역할
<code>uses</code>	마켓플레이스에 공개된 액션을 호출
<code>with</code>	호출한 액션에 입력 파라미터 전달
<code>run</code>	러너 쉘에서 명령을 직접 실행
<code>permissions</code>	실행 토큰에 부여할 최소 권한 선언
<code>if</code>	스텝과 Job의 실행 조건 게이트
<code>workflow_run</code>	선행 워크플로 완료를 트리거로 연결



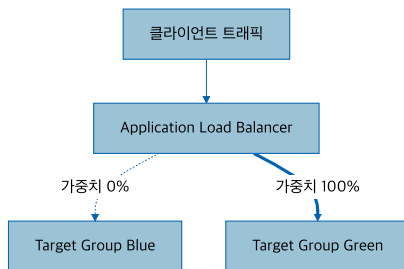
이벤트가 워크플로를 깨우고, 워크플로는 Job을 러너에 배치하며, Job은 Step을 순차 실행합니다. CD 체이닝은 이 계층에서

`workflow_run` 으로 구현됩니다.

다중 인스턴스 ALB Target Group Blue/Green 전환



단일 인스턴스 · 본 실습 구현



다중 인스턴스 · 개념 확장 시나리오

비교 항목	단일 인스턴스 Nginx 프록시	다중 인스턴스 ALB Target Group
전환 주체	호스트 내부 Nginx 프로세스	AWS 관리형 L7 로드밸런서
롤백 방법	<code>service-url.inc</code> 원복 후 리로드	리스너 포워딩 대상 원복
장애 격리	호스트 장애 시 전체 중단	멀티 AZ 분산으로고가용성 확보

☐☐☐ 단일 Target Group 기반 라운드로빈 분산과 달리, Target Group을 2개로 분리한 가중치 전환은 실습 범위 밖의 개념 확장 시나리오입니다.

무중단 배포 전략 비교: Rolling · Blue/Green · Canary

배포 전략	동작 메커니즘	인프라 비용
Rolling Update	구버전 인스턴스를 하나씩 내리고 신버전으로 교체	추가 비용 없음
Blue/Green	신버전 환경을 통째로 기동한 뒤 한 번에 스위칭	전환 순간 2배 리소스
Canary	트래픽의 1~5%만 신버전에 라우팅하고 점진 확대	카나리 전용 소량 추가

✂ Blue/Green의 강점

헬스체크를 통과한 신버전으로 트래픽을 한 번에 넘기므로 구버전과 신버전이 번갈아 노출되는 세션 불일치가 발생하지 않습니다.

⚖ 선택 기준

리소스 여유가 없으면 Rolling, 전환 시점의 일관성이 중요하면 Blue/Green, 신기능의 위험 노출을 최소화하려면 Canary를 선택합니다.

무중단 배포 전략 비교: Rolling · Blue/Green · Canary

배포 전략	롤백 속도와 장애 전과 반경	데이터베이스 스키마 요구
Rolling Update	느림, 배포 도중 신·구 버전 혼재 접근	신·구 버전 동시 서빙 호환성 필수
Blue/Green	매우 빠름, 전환 전 헬스체크로 결함 차단	스위칭 시점의 스키마 충돌 부재
Canary	빠름, 소수 사용자에게만 위험 노출	장기 공존을 견디는 완전한 하위 호환

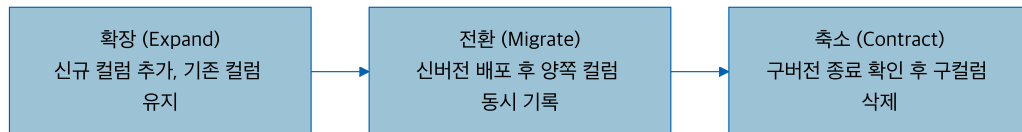
↪ 롤백 속도의 근거

Blue/Green은 구버전 환경이 그대로 남아 있으므로 라우팅 대상만 되돌리면 즉시 복구됩니다. Rolling은 교체된 인스턴스를 다시 순차 배포해야 합니다.

⚠ 장애 전과 반경

전과 반경은 결함 버전에 노출된 사용자 비율로 측정합니다. Canary가 가장 좁고, Rolling은 배포 진행률에 비례해 넓어집니다.

데이터베이스 마이그레이션과 Expand and Contract 패턴



⚠️ 공유 데이터베이스의 제약

Blue와 Green이 같은 데이터베이스를 공유하므로 컬럼 삭제나 이름 변경 같은 파괴적 마이그레이션을 한 번에 실행하면 아직 살아 있는 구버전이 즉시 장애를 일으킵니다.

단계		배포와의 순서
확장		애플리케이션 배포 이전에 선행 적용
축소		구버전 컨테이너 종료를 확인한 뒤 적용

Push 개선형: AWS OIDC와 Systems Manager

비교 항목	Push 기본형	Push 개선형
인증 방식	GitHub Secrets에 pem 프라이빗 키 영구 보관	OIDC 연동으로 단기 IAM 임시 자격 증명 발급
SSH 22번 포트	Runner 접근을 위한 인바운드 개방 필요	완전 차단, AWS 백본망 Session Manager 경유
명령 전달 경로	원격 SSH 셸 세션	Systems Manager Run Command

🔗 OIDC 임시 자격 증명

OIDC (OpenID Connect, 개방형 인증 연동 표준)로 워크플로가 AWS STS에서 유효 시간이 제한된 자격 증명을 발급받으므로 저장소에 고정 키를 두지 않습니다.

📄 SSM Run Command

SSM (AWS Systems Manager, AWS 시스템 관리 서비스) 에이전트가 아웃바운드로 연결을 유지하므로 인바운드 포트를 열지 않고 배포 명령을 전달합니다.

Pull 확장형: CNCF 생태계와 Kubernetes GitOps

구분	CNCF 생태계 위상	GitOps 핵심 역할
Kubernetes	CNCF 최초의 Graduated(졸업) 프로젝트	대규모 컨테이너 분산 오케스트레이션 및 선언형 리소스 관리 표준
Argo CD	2022년 졸업(Graduated)한 CNCF Graduated 프로젝트	Git을 단일 진실 공급원으로 삼는 Kubernetes 네이티브 지속적 배포 표준

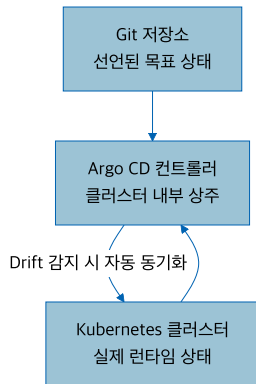
CNCF (클라우드 네이티브 컴퓨팅 재단)

Linux Foundation 산하 기구로, 벤더 종속 없는 개방형 클라우드 표준 기술의 성숙도(Sandbox → Incubating → Graduated)를 공인합니다.

OpenGitOps 4대 원칙

선언형 명세, 불변 버전 관리, 자동 인출(Pull), 지속적 상태 조정(Reconcile)을 통해 인프라와 배포의 신뢰성을 보장합니다.

Kubernetes Argo CD 조정 루프와 자가 치유



비교 항목	Watchtower	Argo CD
감시 대상	단일 호스트의 이미지 다이제스트	Git 선언형 매니페스트 전체
제어 범위	단일 호스트 컨테이너 교체	Pod · Service 등 클러스터 전반
이탈 복구	없음	수동 변조 시 자동 원복 (자가 치유)



조정 루프와 자가 치유 (Self-Healing)

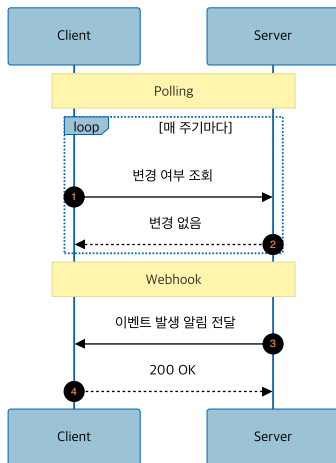
목표 상태와 실제 상태를 상시 대조(Reconciliation)해 관리자의 임의 수정이나 장애 이탈(Drift)을 Git 기준으로 즉시 원복합니다.



클러스터 자격 증명 완벽 격리

외부 CI 러너에 `kubeconfig` 나 관리자 토큰을 전달할 필요가 없으며, 인바운드 포트를 모두 닫은 채 안전하게 대규모 배포를 완결합니다.

Polling과 Webhook의 통신 메커니즘



Polling

변경이 없어도 지속 질의하여 트래픽과 API 한도를 소모하며 반영까지 최대 한 주기만큼 지연됩니다.

Webhook

이벤트 발생 즉시 약속된 URL로 역방향 호출하므로 유휴 트래픽 없이 실시간 처리가 시작됩니다.

판단 기준	선택
수신 측이 인바운드 포트를 열 수 없음	Polling, 주기를 늘려 호출 한도를 관리
반영 지연을 최소화해야 함	Webhook, 수신 엔드포인트에 토큰 인증 적용

Jenkins와 GitHub Actions의 CD 동작 비교

비교 항목	Jenkins	GitHub Actions
인프라 관리	마스터와 에이전트 서버를 자체 구축하고 패치	관리형 클라우드 러너를 즉시 사용
배포 정의	<code>Jenkinsfile</code> 기반 Groovy 파이프라인	선언형 YAML 워크플로
SSH 배포 연동	<code>Publish Over SSH</code> 등 플러그인 설치와 관리	검증된 액션을 <code>uses</code> 로 한 줄 선언
트리거 구조	저장소 웹훅을 받아 젠킨스 서버가 수신	저장소 이벤트가 플랫폼 내부에서 직결
프라이빗 망	사내망에 에이전트를 두어 내부 배포에 유리	Self-hosted Runner를 VPC 내부에 배치



선택 기준

망분리 규제와 전용 하드웨어 연동이 필요하면 Jenkins를 유지합니다.



운영 부담

저장소가 GitHub에 있고 인터넷 연결이 가능하면 러너 운영을 위임합니다.

핵심 기술 면접 질문

Delivery와 Deployment

Continuous Delivery와 Continuous Deployment를 가르는 비즈니스적·기술적 결정 기준은 무엇인가?

단일 호스트 메모리 부담

단일 인스턴스 Nginx Blue/Green 배포에서 메모리 오버헤드를 완화하는 전략은 무엇인가?

SSH 키 등록의 취약점

GitHub Actions에 SSH 프라이빗 키를 등록하는 방식의 보안 취약점과 대안은 무엇인가?

Pull 에이전트의 확장

Watchtower 같은 단일 호스트 Pull 도구가 대규모 오케스트레이션으로 확장되는 원리는 무엇인가?

핵심 기술 면접 질문

헬스체크와 자동 롤백

배포 파이프라인에서 L7 헬스체크와 자동 롤백 메커니즘이 필수적인 이유는 무엇인가?

모범 답변

Delivery와 Deployment

비즈니스 측면에서는 도메인의 규제 요구사항과 릴리스 실패 시의 손실 규모가 기준입니다. 정기 점검 창이 필요하거나 이해관계자의 사전 승인이 필수인 서비스는 수동 승인 게이트를 둔 Continuous Delivery를 채택합니다. 기술 측면에서는 테스트 자동화의 신뢰도가 확보되어 있고 장애 시 수 초 내 이전 버전으로 되돌리는 헬스체크와 롤백 인프라가 갖추어져 있다면 Continuous Deployment를 채택해 피드백 루프를 단축합니다.

단일 호스트 메모리 부담

신규 컨테이너와 기존 컨테이너가 전환 완료까지 수십 초간 메모리를 동시에 점유하므로 OOM (Out of Memory, 메모리 고갈) 위험이 있습니다. `deploy.resources.limits.memory`로 컨테이너별 상한을 명시해 호스트 가용 메모리 초과를 차단하고, `JAVA_TOOL_OPTIONS`의 `-XX:MaxRAMPercentage`로 컨테이너 인지형 JVM이 제한치 안에서만 힙을 쓰도록 강제하며, 리로드 성공 직후 지체 없이 구버전 컨테이너를 정지시킵니다.

모범 답변

SSH 키 등록의 취약점

시크릿에 저장된 SSH 키가 유출되면 EC2 호스트의 관리 계정 권한이 그대로 탈취되고, Runner의 유동 IP를 수용하기 위해 22번 포트를 광범위하게 개방해야 하는 인프라 취약점이 함께 발생합니다. 대안은 OIDC 연동으로 고정 키 없이 단기 IAM 임시 자격 증명을 발급받는 방식, 22번 포트를 닫고 Systems Manager Session Manager로 명령을 전달하는 방식, 프라이빗 서브넷에 Self-hosted Runner를 배치해 내부망 통신으로 배포를 완결하는 방식입니다.

Pull 에이전트의 확장

Watchtower는 호스트의 도커 소켓을 마운트해 이미지 다이제스트를 감시하고 형제 컨테이너를 교체하는 단일 호스트 Pull 에이전트입니다. 이 철학이 선언형 인프라와 대규모 분산 클러스터로 확장된 것이 CNCF의 졸업(Graduated) 프로젝트인 Kubernetes와 Argo CD 기반 GitOps입니다. Argo CD는 클러스터 내부 컨트롤러로 상주하며 Git 매니페스트(Desired State)와 런타임 상태(Actual State)를 지속 대조(Reconciliation Loop)하고 Drift를 자동 복구(Self-Healing)합니다. 외부 CI 러너에 kubeconfig 관리자 자격 증명을 노출하지 않고 인바운드 포트를 완전 차단한 채 엔터프라이즈급 보안 배포를 완결합니다.

모범 답변

헬스체크와 자동 롤백

프로세스가 백그라운드에 기동되었다는 사실만으로는 애플리케이션이 실제 트래픽을 처리할 수 있는 상태임을 보장하지 못합니다. Spring Boot는 DB 커넥션 풀 연결 지연, 엔티티 매핑 오류, 필수 환경변수 누락 상황에서도 프로세스는 살아 있는 채 HTTP 500을 반환할 수 있습니다. 따라서 배포 스크립트는 프록시의 라우팅 대상을 전환하기 전에 신규 컨테이너의 포트로 실제 HTTP 엔드포인트를 반복 폴링해 200 응답을 명시적으로 검증해야 합니다. 지정 횟수 안에 정상 응답을 얻지 못하면 신규 컨테이너를 즉시 중단하고 기존 활성 상태를 보존하는 자동 롤백을 수행해야 서비스 장애를 원천 차단할 수 있습니다.

깊이 알아보기

Nginx 신호 제어와 무중단 리로드

<https://nginx.org/en/docs/control.html>

Watchtower 공식 문서

<https://containrrr.dev/watchtower/>

Argo CD 공식 문서

<https://argo-cd.readthedocs.io/en/stable/>

Slack Incoming Webhooks

<https://api.slack.com/messaging/webhooks>

깊이 알아보기

GitHub Actions 워크플로 트리거 이벤트

<https://docs.github.com/en/actions/using-workflows/events-that-trigger-workflows>

GitHub Actions 시크릿 사용

<https://docs.github.com/en/actions/security-for-github-actions/security-guides/using-secrets-in-github-actions>

SSH 원격 실행 액션 appleboy/ssh-action

<https://github.com/appleboy/ssh-action>

CNCF 공식 포털

<https://www.cncf.io/>



- EOF -